

C Preprocessor: #define Directives

Understanding how to use #define for constants and functional macros



Constants

Using #define to create constants



Functional Macros

Creating macros that work like functions



Cautions

Common pitfalls and how to avoid them





Customizing Language Syntax

The preprocessor allows us to customize the language. For example to replace { and } of C language to **begin** and **end** as block-statement delimiters (as like the case in PASCAL) we can achieve this by writing:

```
#define begin {  
#define end }
```

During compilation all occurrences of **begin** and **end** get replaced by corresponding { and }. So the subsequent C compilation stage does not know any difference!





#define for Constants

#define is used to define constants. The syntax is:

```
#define <literal> <replacement-value>
```

literal is identifier which is replaced with replacement-value in the program.

Examples:

```
#define MAXSIZE 256  
#define PI 3.142857
```

The C preprocessor simply searches through the C code before it is compiled and replaces every instance of MAXSIZE with 256.





More Examples of #define Constants

```
#define FALSE 0  
#define TRUE !FALSE
```

The literal TRUE is substituted by !FALSE and FALSE is substituted by the value 0 at every occurrence, before compilation of the program. Since the values of the literal are constant throughout the program, they are called as constant.

#define can be rewritten as:

```
#define <constant-name> <replacement-value>
```





Examples and Syntax Notes

Let us consider few examples:

```
#define M 5  
#define SUBJECTS 6  
#define PI 3.142857  
#define COUNTRY INDIA
```



Important Note:

No semicolon (;) need to be placed as the delimiter at the end of a `#define` line. This is just one of the ways that the syntax of the preprocessor is different from the rest of C statements (commands).



⚠ Caution: Semicolons in #define

If you unintentionally place the semicolon at the end as below:

```
#define MAXLINE 100; /* WRONG */
```

and if you declare as shown below in the declaration section,

```
char line[MAXLINE];
```

the preprocessor will expand it to:

```
char line[100;]; /* WRONG */
```

Always remember: **No semicolon at the end of #define statements!**



⚙️ #define to Create Functional Macros

One of the powerful functions of the CPP is the ability to simulate functions using parameterized macros.

For example, we might have some code to square a number:

```
int square(int x)
{
    return x * x;
}
```

We can rewrite the above code using a macro as follows:

```
#define square(x) ((x) * (x))
```



Macros with arguments must be defined using the `#define` directive before they can be used. The argument list is enclosed in parentheses and must immediately follow the macro name. Spaces are not allowed between the macro name and open parenthesis.

7/10

Example 1: MAX Macro

```
/* Program to find the maximum of two numbers */
#include <stdio.h>

#define MAX(x,y) ((x) > (y) ? (x) : (y))

int main(void)
{
    printf("Max between 20 and 10 is %d\n", MAX(10, 20));
    return 0;
}
```

Output:

Max between 20 and 10 is 20.

8/10

Example 2: SQUARE Macro

```
/* Program to find the square of a number using macro */
#include <stdio.h>

#define SQUARE(x) (x*x)

main()
{
    int v,y;
    printf("Enter any number to find its square: ");
    scanf("%d", &v);
    y = SQUARE(v);
    printf("\nThe square of %d is %d", v, y);
}
```

Output:

```
Enter any number to find its square: 10
The square of 10 is 100
```

9/10

Caution When Using Macros

It should be very careful in using Macros. In particular the textual substitution means that arithmetic expressions are liable to be corrupted by the order of evaluation rules (precedence rules).

Here is an example of a macro, which won't work:

```
#define DOUBLE(n) n + n
```

Now if we have a statement,

```
z = DOUBLE(p) * q;
```

This will be expanded to



```
z = p + p * q;
```

And since `*` has a higher priority than `+`, the compiler will treat it as:

```
z = p + (p * q);
```

Solution:

The problem can be solved using a more robust definition of `DOUBLE`.

```
#define DOUBLE(n) (n + n)
```

Here, the braces around the definition force the expression to be evaluated before any surrounding operators are applied.

